

Mini GDD - Microjuego 2D (Space Shooter)

Nombre, Apellido: Yating, Sun

Nº matrícula: 240035

1. Concepto e Idea General

El juego es un Arcade Space Shooter en 2D inspirado en los clásicos de esquivar y destruir meteoritos. El objetivo del jugador es aguantar el mayor tiempo posible destruyendo asteroides para sumar la máxima puntuación antes de colisionar contra uno. Al ser destruido, cada asteroide se fragmenta en pequeños trozos y activa un efecto de temblor en la cámara para mejorar el impacto visual.

2. Configuración de Escenas y Cámara

El proyecto cuenta con una escena principal denominada SampleScene:

- **Cámara (Main Camera):** Configurada en proyección Orthographic. Cuenta con el script Camera Shake.cs adjunto, el cual genera una pequeña vibración de la cámara cuando una bala impacta contra un asteroide.
- **Sistema de Teletransporte / Envolvente (Screen Wrapping):** Para evitar que la nave se salga de los límites jugables, el script del jugador comprueba constantemente su posición respecto a dos variables ($xBorderLimit = 9f$ e $yBorderLimit = 5f$). Si la nave cruza cualquiera de los cuatro bordes de la pantalla, reaparece automáticamente en el lado opuesto.

3. GameObjects, Componentes y Prefabs

He organizado los objetos en la escena utilizando Tags y Layer Mask para gestionar correctamente las colisiones e interacciones:

A. Player (Nave)

- **Rigidbody:** Configurado en modo 3D para aplicar fuerzas de propulsión.
- **Collider:** Utilizado para detectar colisiones físicas contra los meteoritos.
- **Objeto hijo (Gun):** Un transform ubicado en la parte delantera de la nave que sirve como punto de origen (firingPoint) para la creación de las balas.

- **Tag:** Player.

B. Prefab: Meteor (Enemigo)

- **SpriteRenderer:** Renderiza la imagen Meteor_0 utilizando el material Sprite-Unlit-Default.
- **Rigidbody (3D):** Componente de físicas 3D con masa 1, Angular Damping en 0.05 y la casilla Use Gravity activada. Recibe la velocidad descendente (linearVelocity) y la rotación impulsiva (AddTorque) calculadas en el script EnemySpawner.cs.
- **Sphere Collider:** Colisionador 3D esférico descentrado (Center: X: 0, Y: -2, Z: 0) con un radio de 1 para ajustar con precisión el área de contacto física con la nave y las balas.
- **Tag y Layer:** Asignado con la etiqueta Enemy en la capa Default.

C. Prefab: Bullet (Bala)

- Avanza en la dirección hacia la que apuntaba la nave (targetVector). Cuenta con autodestrucción por tiempo (maxLifeTime = 3f) y genera la fragmentación del asteroide al colisionar.

D. Interface de Usuario (Canvas / UI)

- Un componente de texto clásico (UnityEngine.UI.Text) etiquetado con el Tag "UI", que muestra la puntuación acumulada en tiempo real (Puntos : X).

E. Prefab: SmallAsteroid (Fragmento):

- Generado automáticamente en grupo (fragmentCount = 5) cuando una bala destruye un asteroide grande. Recibe un impulso de fuerza aleatorio en todas direcciones (AddForce) y se autodestruye tras 1.5 segundos.

4. Mecánicas de Juego Detalladas

- Control de Propulsión y Rotación:
 - Lee los ejes de entrada personalizados "Rotate" y "Thrust".
 - La rotación se aplica sobre el eje Z con transform.Rotate(Vector3.forward, ...).
 - La aceleración aplica una fuerza con _rigid.AddForce() usando la dirección derecha de la nave (transform.right).
- Mecánica de Disparo y Fragmentación:
 - Al pulsar la tecla Espacio, se instancia la bala en la posición del objeto gun y se le asigna la dirección actual de la nave (transform.right).
 - Al impactar contra un enemigo (OnCollisionEnter), la bala llama a la

función `ShatterAsteroid()`, instanciando 5 fragmentos (`smallAsteroidPrefab`), suma un punto a la variable global `Player.SCORE`, actualiza la UI, activa el temblor de cámara (`CameraShake`) y destruye tanto la bala como el asteroide.

- **Generación Progresiva de Asteroides (Spawning):**
 - El objeto `EnemySpawner` controla la aparición mediante un temporizador en `Update()`.
 - La frecuencia de aparición (`spawnRatePerMinute`) aumenta gradualmente con el tiempo (`spawnRateIncrement`), haciendo el juego cada vez más difícil. Genera los asteroides en una posición X aleatoria (`Random.Range(-xLimit, xLimit)`) por encima de la pantalla.
- **Sistema de Puntuación y Muerte:**
 - Cada meteorito grande destruido por una bala incrementa `Player.SCORE` en 1.
 - Si un asteroide colisiona directamente contra la nave del jugador, la puntuación se reinicia a 0 (`SCORE = 0`) y se recarga la escena actual (`SceneManager.LoadScene`) de inmediato.

5. Explicación de los Scripts (C#)

1. Player.cs

- **Variables principales:**
 - `public float thrustForce = 100f`: Fuerza de aceleración.
 - `public float rotationSpeed = 120f`: Velocidad de giro.
 - `public float xBorderLimit, yBorderLimit`: Límites para el Screen Wrapping.
 - `public GameObject gun, bulletPrefab`: Referencias al punto de disparo y al prefab de la bala.
 - `public static int SCORE`: Variable estática para almacenar la puntuación global.
- **Métodos clave:**
 - `Update()`: Procesa la rotación, la fuerza de empuje, el límite de pantalla y el disparo al presionar Espacio.
 - `OnCollisionEnter(Collision collision)`: Detecta el choque contra objetos etiquetados como "Enemy". Reinicia la puntuación y la escena.

2. Bullet.cs

- **Variables principales:**
 - `public float speed = 10f, maxLifeTime = 3f`: Velocidad y tiempo de vida del proyectil.
 - `public Vector3 targetVector`: Dirección de movimiento asignada al nacer.

- public GameObject smallAsteroidPrefab: Prefab de los fragmentos pequeños.
- public int fragmentCount = 5: Cantidad de trozos creados tras el impacto.
- **Métodos clave:**
 - Start(): Destruye el proyectil automáticamente tras maxLifeTime.
 - Update(): Mueve la bala con transform.Translate.
 - OnCollisionEnter(): Detecta colisión con "Enemy", destruye el asteroide, invoca ShatterAsteroid(), suma puntos, actualiza el texto UI y activa CameraShake.
 - ShatterAsteroid(): Genera fragmentos saliendo disparados en direcciones aleatorias mediante AddForce.
 - UpdateScoreText(): Busca el objeto UI con Tag "UI" y actualiza el componente Text.

3. EnemySpawner.cs

- **Variables principales:**
 - public GameObject asteroidPrefab: Prefab del asteroide grande.
 - public float spawnRatePerMinute = 30f: Frecuencia inicial de spawn.
 - public float spawnRateIncrement = 1f: Incremento de dificultad progresivo.
 - public float xLimit = 8f, maxTimeLife = 6f, asteroidSpeed = 2f.
- **Métodos clave:**
 - Update(): Evalúa el tiempo transcurrido (Time.time > spawnNext). Calcula la posición aleatoria superior, instancia el asteroide, le aplica velocidad e impulso de rotación aleatorio (AddTorque) y programa su autodestrucción.

4. CameraShake.cs

- **Variables principales:**
 - public static CameraShake Instance: Patrón Singleton para acceso global rápido.
- **Métodos clave:**
 - Shake(float duration, float magnitude): Inicia la corrutina de temblor.
 - DoShake(): Corrutina que desplaza aleatoriamente la posición local de la cámara durante un breve lapso de tiempo y la regresa a su posición original al finalizar.